This class can then be provided to DailySummary as a collaborator using simple dependency injection:

15-using-test-doubles-effectively/04/daily_summary_email/lib/daily_summary.rb

```
class DailySummary
  def initialize(email_sender: EmailSender.new)
    @email_sender = email_sender
  end

  def send_daily_summary(user_email, todays_messages)
    message_count = todays_messages.count
    thread_count  = todays_messages.map { |m| m[:thread_id] }.uniq.count
    subject       = 'Your daily message summary'
    body          = "You missed #{message_count} messages " \
                    "in #{thread_count} threads today"

    @email_sender.deliver(email: user_email, subject: subject, body: body)
  end
end
```

We're not suggesting you break up classes like this just to satisfy a "don't stub the subject" rule. Rather, we're saying that this guideline has told us something about the code. If we put the mail-delivery logic into its own class, we gain the following benefits:

- We can use EmailSender to send other emails besides daily summaries.
- We can test the SMTP logic independently, apart from any specific email.
- We now have a single place to add other email features such as email subscription preferences.

Once the refactoring is done, we no longer need to fake out methods on DailySummary itself. Instead, we can spin up a verifying double for the EmailSender:

15-using-test-doubles-effectively/04/daily_summary_email/spec/daily_summary_spec.rb

```
it "sends a summary of today's messages and threads" do
  email_sender = instance_double(EmailSender)
  summary = DailySummary.new(email_sender: email_sender)

  expect(email_sender).to receive(:deliver).with(
    hash_including(body: 'You missed 3 messages in 2 threads today')
  )

  summary.send_daily_summary('user@example.com', todays_messages)
end
```

With this change, the boundary between the constructed test environment (the fake EmailSender) and the object we are testing (the DailySummary) is much clearer. A code smell in our specs has given us information we can use to improve our design.